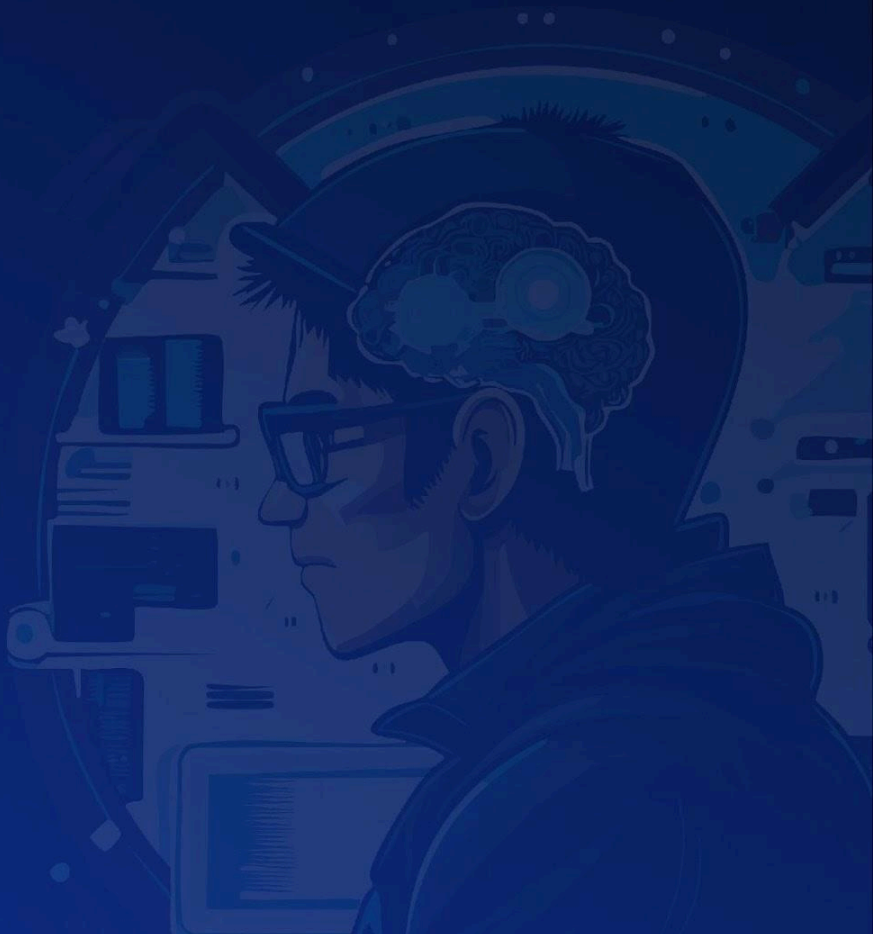
Cypher Launchpad

# Security Review

# Disclaimer

## Security Review

Cypher Launchpad



# Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

# Table of Contents

## Security Review

Cypher Launchpad

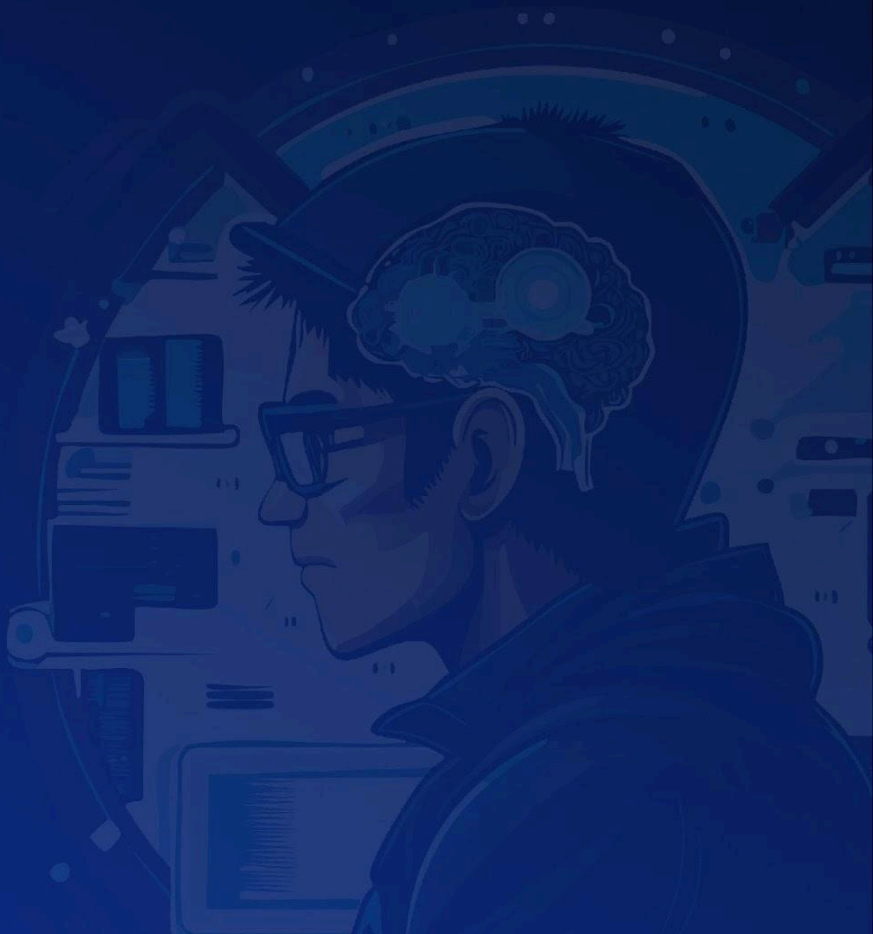
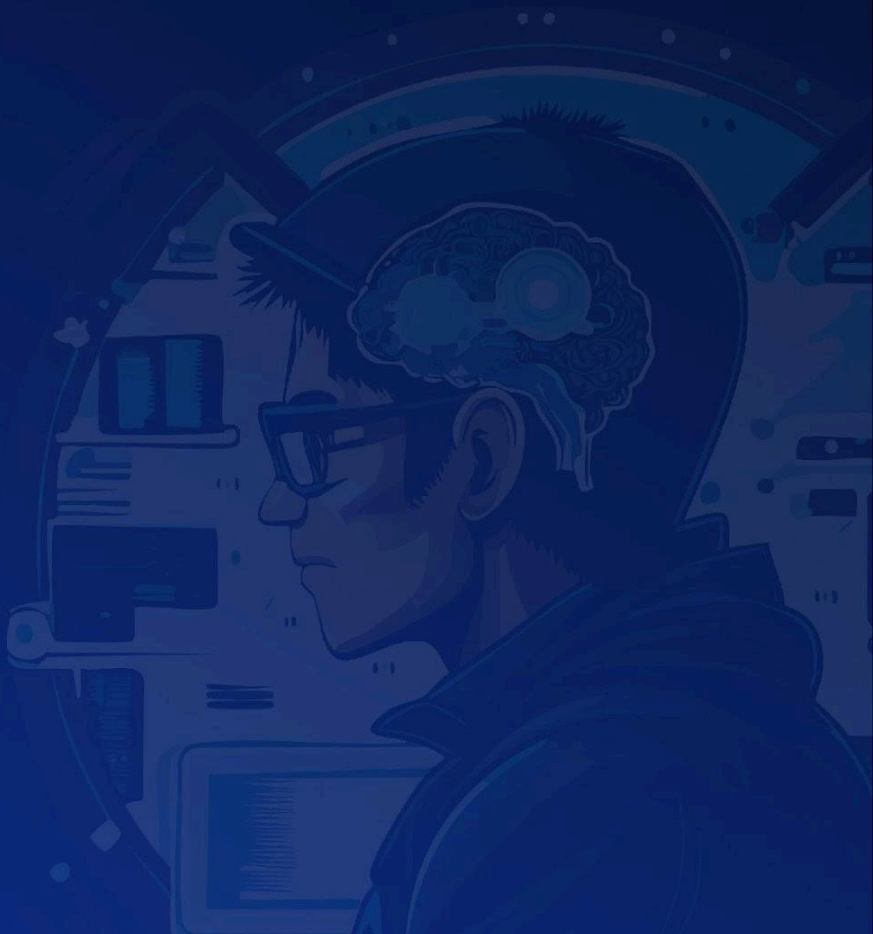


Table of Contents

# Summary

## Security Review

Cypher Launchpad



# Summary

Three Sigma audited Cypher in a 1.2 person week engagement. The audit was conducted from 11/02/2026 to 13/02/2026.

## Protocol Description

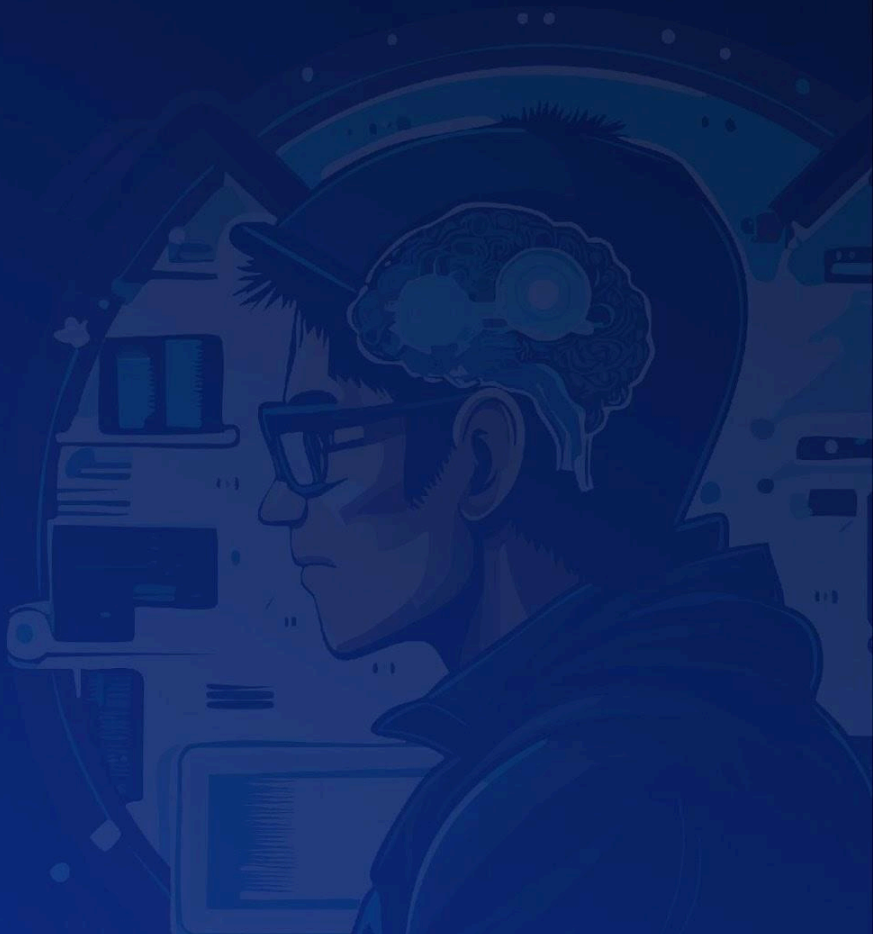
Cypher Factory is an Ethereum Mainnet token factory built on top of the Cypher DEX. Whenever a new token is launched, a new Algebra V4 liquidity pool is created, setting its `sqrtPriceX96` and it enters a _Bonding Curve_ phase where its params are inherited from the BCTokenFactory.sol config. During the _Bonding Curve_ phase, users are able to trade within a virtual bonding curve until the token `MAX_SUPPLY` is reached (which is equivalent to reaching a target market cap or `TARGET_ETH`).

Once the `MAX_SUPPLY` is reached, all the ETH available plus an initial BCToken supply are deployed as liquidity to the created LP and the positions are transferred to a Harvester contract which locks them but enables a privileged account or contract to harvest the fees. BCToken supply is deployed from the _seed tick_ to MAX or MIN tick and ETH is deployed from _seed tick_ to the _initial expected tick_ which matches the first price of the Bonding Curve.

# Scope

## Security Review

Cypher Launchpad



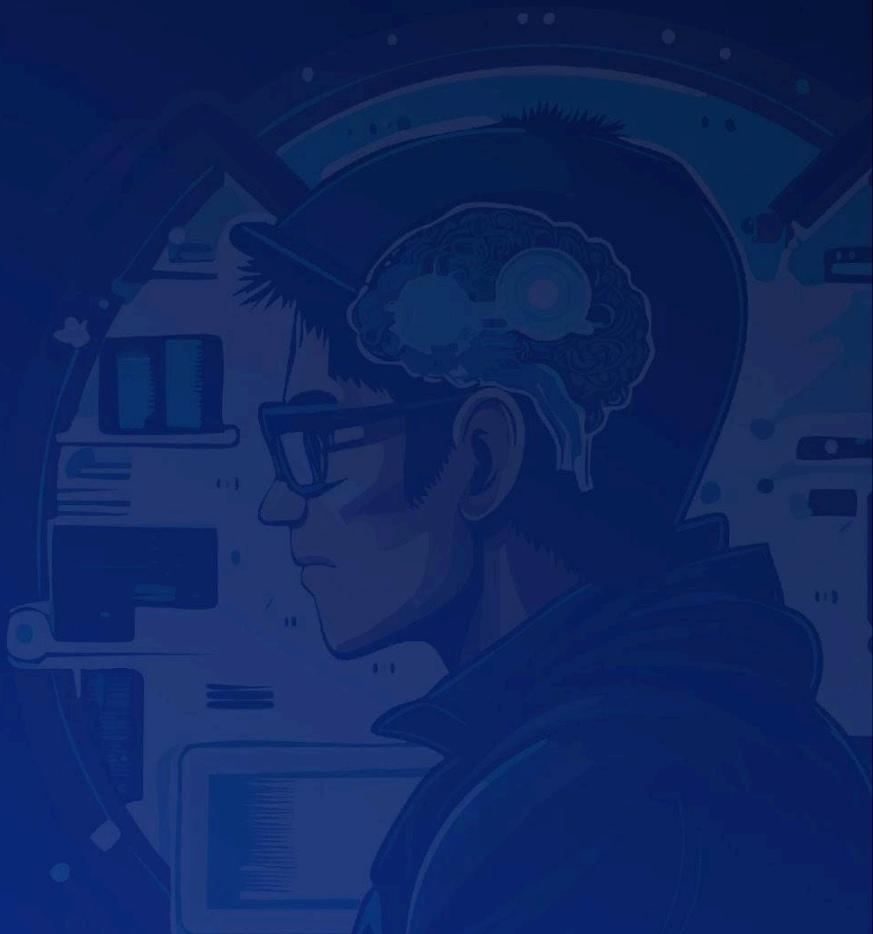
# Scope

| Filepath | nSLOC |
|---|---|
| src/algebra/plugin/AlgebraFeePluginV1.sol | 99 |
| src/algebra/plugin/AlgebraFeePluginFactoryV1.sol | 169 |
| src/BCToken.sol | 151 |
| src/LiquidityManager.sol | 152 |
| src/BCTokenFactory.sol | 181 |
| src/BCTokenDeployer.sol | 25 |
| **Total** | **777** |

# Methodology

## Security Review

Cypher Launchpad



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

## Taxonomy

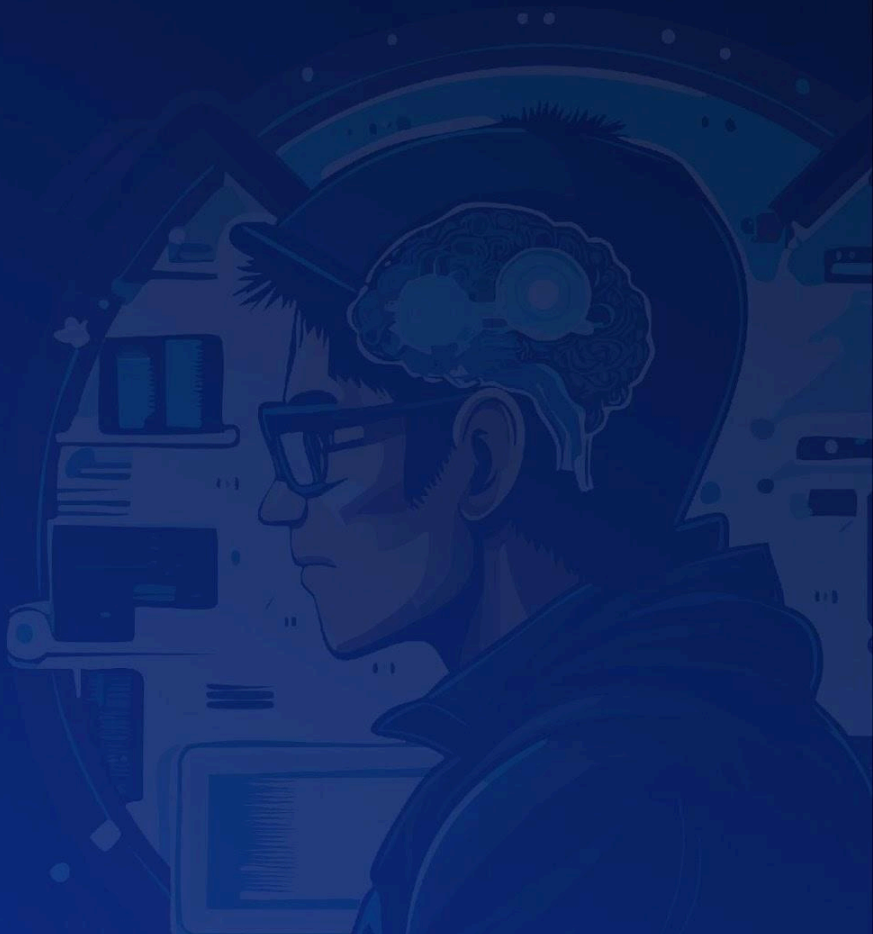
In this audit, we classify findings based on Immunefi's Vulnerability Severity Classification System (v2.3) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

| Impact / Likelihood | LOW | MEDIUM | HIGH |
|---|---|---|---|
| NONE | None | | |
| LOW | Low | | |
| MEDIUM | Low | Medium | Medium |
| HIGH | Medium | High | High |
| CRITICAL | High | Critical | Critical |

# Project Dashboard

## Security Review

Cypher Launchpad



# Project Dashboard

## Application Summary

| | |
|---|---|
| Name | Cypher |
| Repository | https://github.com/CypherIndustries/factory-contracts |
| Commit | e3694ce |
| Language | Solidity |
| Platform | Ethereum Mainnet |

## Engagement Summary

| | |
|---|---|
| Timeline | 11/02/2026 to 13/02/2026 |
| № of Auditors | 2 |
| Review Time | 1.2 person weeks |

## Vulnerability Summary

| Issue Classification | Found | Addressed | Acknowledged |
|---|---|---|---|
| Critical | 0 | 0 | 0 |
| High | 0 | 0 | 0 |
| Medium | 3 | 3 | 0 |
| Low | 4 | 4 | 0 |
| None | 3 | 2 | 1 |

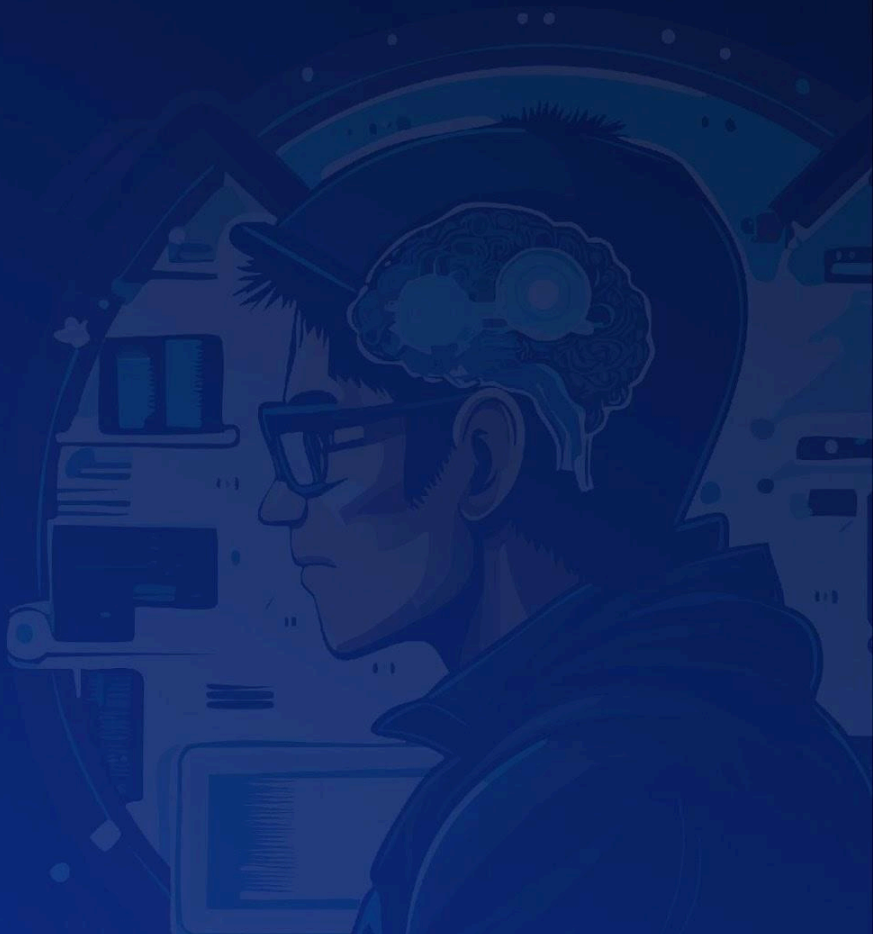
## Category Breakdown

| | |
|---|---|
| Suggestion | 3 |
| Documentation | 0 |
| Bug | 7 |
| Optimization | 0 |
| Good Code Practices | 0 |

# Risk Section

## Security Review

Cypher Launchpad



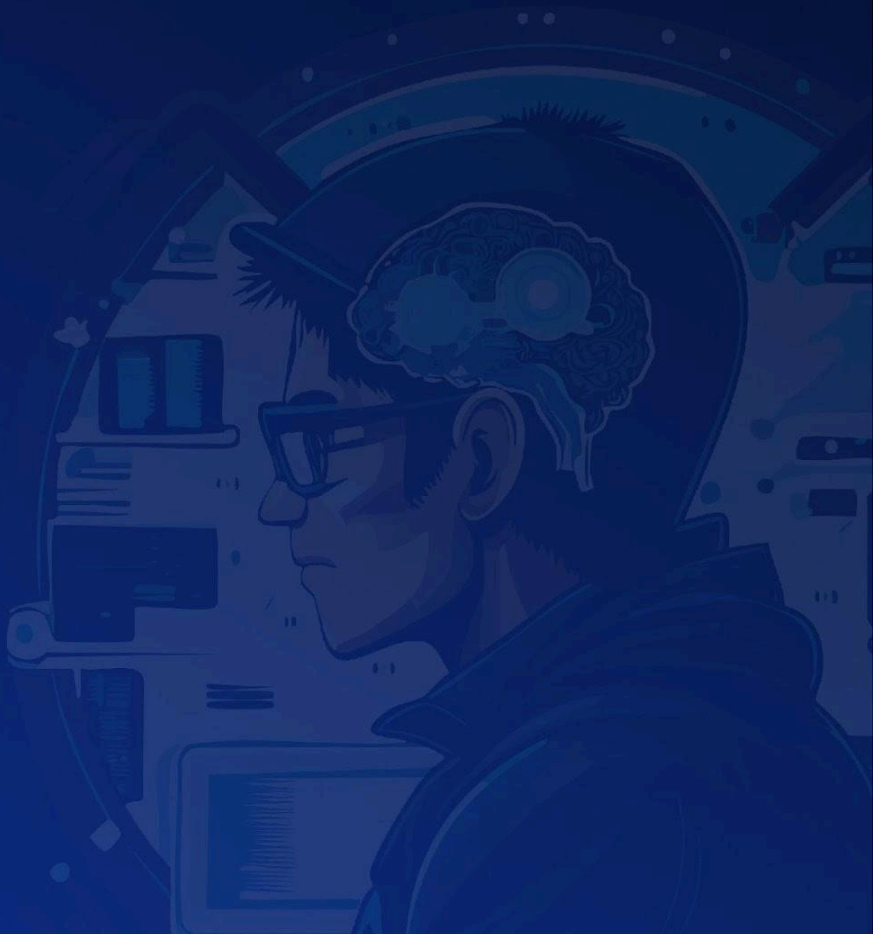
# Risk Section

- **Global Protocol Fee**: The factoryOwner of AlgebraFeePluginFactoryV1 can call setProtocolFeeRatio() at any time to retroactively increase the protocol fee share (up to 100%) across all existing pools.

# Findings

## Security Review

Cypher Launchpad



# Findings

## 3S-Launchpad-M01

Stale **pluginFactory** reference in **LiquidityManager::seedLP** leads to permanent inability to graduate pre-existing BCTokens

| | |
|---|---|
| Id | 3S-Launchpad-M01 |
| Classification | Medium |
| Impact | High |
| Likelihood | Low |
| Category | Bug |
| Status | Addressed in #fb8abe5. |

---

### Description

**LiquidityManager** manages pool creation and liquidity seeding for BCTokens. When a BCToken is deployed, its constructor calls **LiquidityManager::createAndInitializePool**, which uses the current **pluginFactory** to deploy the Algebra pool. The pool address is deterministically derived from the deployer address (i.e., the **pluginFactory**) among other parameters. Later, when the BCToken reaches **MAX_SUPPLY** and graduates, **BCToken::_seedLP** calls **LiquidityManager::seedLP**, which passes the current **pluginFactory** value as the **deployer** field in the **MintParams** struct.

The **pluginFactory** state variable can be updated at any time by the owner via **LiquidityManager::setPluginFactory**. If this occurs between a BCToken's pool creation and its graduation, **seedLP** will reference the new **pluginFactory** address as the **deployer**. Because the new **pluginFactory** did not deploy the pool, **PositionManager::mint** will derive a pool address with no deployed contract code, causing the transaction to revert. Since **seedLP** is the only path to set **isLPd = true**, the affected BCToken is permanently stuck: it cannot graduate, transfers remain restricted, and all ETH and tokens held by the contract become locked.

#### Steps to Reproduce

1. The current **pluginFactory** is address A. A user creates a BCToken via **BCTokenFactory**. During construction, **LiquidityManager::createAndInitializePool** deploys the Algebra pool using address A as the deployer.

2. The owner calls **LiquidityManager::setPluginFactory** and updates **pluginFactory** to address B.

3. The BCToken reaches **MAX_SUPPLY**, triggering **BCToken::_seedLP**, which calls **LiquidityManager::seedLP**.

4. Inside **seedLP**, both **_mintPosition** calls pass **deployer: pluginFactory** (now address B) in **MintParams**.

5. **PositionManager::mint** derives the pool address using address B. Since address A originally deployed the pool, the derived address contains no contract code, and the transaction reverts.

6. The BCToken cannot graduate. All ETH and tokens remain permanently locked in the contract.

---

### Recommendation

Store the **pluginFactory** address used during pool creation for each BCToken, and reference it during **seedLP** instead of the current **pluginFactory** value.

```
+ /// @dev Tracks the pluginFactory that deployed the pool for each BCToken
+ mapping(address => address) public tokenDeployer;
  function createAndInitializePool(
      uint256 virtualReserves,
      uint256 initialSupply,
      uint256 maxSupply,
      address creator,
      uint8 communityFeeRatio,
      IBCTokenFactory.StakingConfig calldata stakingConfig
  ) external returns(uint256 targetETH, address pool) {
      (address token0, address token1) =
          msg.sender < WETH ? (msg.sender, WETH) : (WETH, msg.sender);
      targetETH = ((maxSupply - initialSupply) * virtualReserves) / initialSupply +
virtualReserves;
      _checkIfLiquidityPoolExists(token0, token1);
      uint160 seedSqrtX96Price = _computeAndValidateTicks(
          msg.sender, token0, virtualReserves, initialSupply, maxSupply, targetETH
      );
      pool = IAlgebraFeePluginFactory(pluginFactory).createCustomPool(
          creator, token0, token1, "", communityFeeRatio, stakingConfig
      );
+     tokenDeployer[msg.sender] = pluginFactory;
      IAlgebraPool(pool).initialize(seedSqrtX96Price);
      return (targetETH, pool);
```

```
}
```

Then in **seedLP**, replace **pluginFactory** with the stored deployer:

```
function seedLP(uint256 bcTokenBalance, uint256 wethBalance) external {
    address token = msg.sender;
    IBCToken coin = IBCToken(token);
+    address deployer = tokenDeployer[token];
    // ... (unchanged) ...
    _mintPosition(IPositionManager.MintParams({
        token0: token0,
        token1: token1,
-       deployer: pluginFactory,
+        deployer: deployer,
        tickLower: token0LowTick,
        tickUpper: token0HighTick,
        amount0Desired: amount0,
        amount1Desired: 0,
        amount0Min: 0,
        amount1Min: 0,
        recipient: harvester,
        deadline: block.timestamp
    }));
    _mintPosition(IPositionManager.MintParams({
        token0: token0,
        token1: token1,
-       deployer: pluginFactory,
+        deployer: deployer,
        tickLower: token1LowTick,
        tickUpper: token1HighTick,
        amount0Desired: 0,
        amount1Desired: amount1,
        amount0Min: 0,
        amount1Min: 0,
        recipient: harvester,
        deadline: block.timestamp
    }));
}
```

# 3S-Launchpad-M02

Missing access control and input validation in **TokenVesting::createVestingSchedule** allows bypassing **BCToken** pre-LP transfer restrictions

| | |
|---|---|
| Id | 3S-Launchpad-M02 |
| Classification | Medium |
| Impact | Medium |
| Likelihood | High |
| Category | Bug |
| Status | Addressed in #1431d0b. |

---

### Description

**BCToken** enforces transfer restrictions before liquidity is seeded (i.e., while **isLPd** is **false**). Both **BCToken::transfer** and **BCToken::transferFrom** revert unless at least one participant (sender, receiver, or **msg.sender**) is present in the **transferWhitelist** mapping. This restriction prevents open token circulation during the bonding curve phase.

During token deployment, **BCTokenFactory::_deployToken** automatically adds the **TokenVesting** contract to each new **BCToken**'s **transferWhitelist**:

```
address _tokenVesting = tokenVesting;
if (_tokenVesting != address(0)) {
    IBCToken(token).setTransferWhitelist(_tokenVesting, true);
}
```

The **TokenVesting::createVestingSchedule** function has no access control, meaning any user can create a vesting schedule for any **BCToken**. Additionally, the function does not validate that the **start** timestamp is not in the past or that the **duration** meets a minimum threshold. This allows an attacker to create a vesting schedule with a **start** time in the past and a minimal **duration** (e.g., 1 second) with **cliff** set to **0**, resulting in the entire vested amount being immediately releasable.

Because the **TokenVesting** contract is whitelisted, **safeTransferFrom** in **createVestingSchedule** succeeds (the **TokenVesting** address satisfies the **transferWhitelist[to]** and **transferWhitelist[msg.sender]** checks in **BCToken::transferFrom**). When the beneficiary calls **TokenVesting::release**,

**safeTransfer** sends tokens from the whitelisted **TokenVesting** contract, satisfying the **transferWhitelist[msg.sender]** check in **BCToken::transfer**. This effectively allows any holder to transfer **BCToken** to any arbitrary address before LP is seeded, bypassing the transfer restriction entirely.

---

### Recommendation

Add access control to **TokenVesting::createVestingSchedule** so that only authorized addresses can create vesting schedules. Additionally, enforce minimum bounds on vesting parameters to prevent trivially short or retroactive schedules:

```
+ /// @notice Minimum vesting duration
+ uint256 public constant MIN_DURATION = 7 days;
+
+ /// @notice Authorized schedule creators
+ mapping(address => bool) public schedulers;
  function createVestingSchedule(
      address[] calldata beneficiaries,
      address token,
      uint256 start,
      uint256 cliff,
      uint256 duration,
      uint256 slicePeriodSeconds,
      uint256[] calldata amounts
  ) external returns (bytes32[] memory vestingScheduleIds) {
+     if (!schedulers[msg.sender]) {
+         revert UnauthorizedCaller(msg.sender, address(0));
+     }
+     if (start < block.timestamp) {
+         revert InvalidStartTime();
+     }
+     if (duration < MIN_DURATION) {
+         revert DurationTooShort();
+     }
      uint256 beneficiariesLength = beneficiaries.length;
```

# 3S-Launchpad-M03

Stale owner snapshot in **BCToken** at deployment leads to fragmented ownership and misdirected protocol fees after factory ownership transfer

| | |
|---|---|
| Id | 3S-Launchpad-M03 |
| Classification | Medium |
| Impact | Medium |
| Likelihood | Medium |
| Category | Bug |
| Status | Addressed in #d231514. |

## Description

When **BCTokenFactory::_deployToken** creates a new **BCToken**, it passes **owner()** as the **_delegate** parameter (**BCTokenFactory.sol:173**). The **BCToken** constructor then sets this value as its own **owner** via **Ownable(params._delegate)** (**BCToken.sol:86**) and as the LayerZero delegate through the **OFT** → **OAppCore** inheritance chain. This captures the factory owner's address as a point-in-time snapshot rather than maintaining a live reference.

If **BCTokenFactory** ownership is subsequently transferred via **Ownable::transferOwnership**, all previously deployed **BCToken** instances retain the old owner address. This creates two concrete problems. First, **BCToken::_seedLP** distributes protocol fees to **owner()** (**BCToken.sol:256**), meaning the old factory owner continues to receive protocol fees from tokens that have not yet completed their bonding curve, while the new factory owner receives nothing. Second, the old owner retains privileged access to **BCToken**-level configuration such as **setTransferWhitelist** and LayerZero endpoint settings through the **OApp** delegate, which the new factory owner cannot control or revoke.

The result is a permanent split in protocol authority: the new factory owner controls factory-level operations (fee parameters, bonding curve, pausing), while the old owner controls per-token operations and fee collection for every **BCToken** deployed before the ownership transfer.

## Recommendation

For protocol fee distribution, **BCToken::_seedLP** should query the factory's current owner dynamically instead of relying on the stale local **owner()**:

```
-   IERC20(weth).transfer(owner(), ethBalance * PROTOCOL_FEE / 10_000);
+   IERC20(weth).transfer(IBCTokenFactory(FACTORY).owner(), ethBalance *
PROTOCOL_FEE / 10_000);
```

For the LayerZero delegate desync, override **transferOwnership** in **BCToken** to call **OAppCore::setDelegate** before the ownership change takes effect. Since both **transferOwnership** and **setDelegate** are **onlyOwner**, the caller is already verified and **setDelegate** can execute while **msg.sender** is still the current owner:

```
+ function transferOwnership(address newOwner) public override onlyOwner {
+     setDelegate(newOwner);
+     super.transferOwnership(newOwner);
+ }
```

# 3S-Launchpad-L01

## Unrestricted access to **LiquidityManager::createAndInitializePool** allows creation of illegitimate pools

| | |
|---|---|
| Id | 3S-Launchpad-L01 |
| Classification | Low |
| Impact | Low |
| Likelihood | Low |
| Category | Bug |
| Status | Addressed in #831a5ac. |

### Description

**LiquidityManager::createAndInitializePool** is responsible for deploying and initializing an Algebra pool that pairs the calling BCToken with WETH. It is designed to be invoked exclusively by a **BCToken** contract during its constructor, as part of the token deployment flow initiated through **BCTokenFactory::deploy**. The function uses **msg.sender** directly as the token address for one side of the pool pair.

However, the function is declared **external** without any caller validation. Any externally owned account or arbitrary contract can invoke it, passing chosen **virtualReserves**, **initialSupply**, **maxSupply**, and **stakingConfig** parameters. Since **msg.sender** is treated as the token address, the caller's own address would be used to create and initialize a pool against WETH in the Algebra plugin factory. While the **_checkIfLiquidityPoolExists** guard prevents duplicate pools for the same pair, a single successful call from an unauthorized address is sufficient to create a pool that the protocol did not sanction. This can pollute on-chain state and events that frontends or indexers may consume, leading to the surfacing of illegitimate pools as if they were protocol-originated.

### Recommendation

Add a reference to **BCTokenFactory** in **LiquidityManager** and validate that **msg.sender** is a registered BCToken. To handle the fact that **createAndInitializePool** is called during the BCToken constructor (before the factory sets **bcTokens[token] = true**), move the registration in **BCTokenFactory::_deployToken** to occur before the actual deployment call, since the token address is already known via CREATE3 precomputation at that point.

In **BCTokenFactory::_deployToken**, register the token before deploying:

```
  token = TOKEN_DEPLOYER.getDeployed(deploymentSalt);
  if (token.code.length > 0) revert SaltAlreadyUsed();
+ bcTokens[token] = true;
  TOKEN_DEPLOYER.deploy(
```

Store the factory address in **LiquidityManager** and enforce the check in **createAndInitializePool**:

```
+ address public factory;
```

```
  function createAndInitializePool(
      uint256 virtualReserves,
      uint256 initialSupply,
      uint256 maxSupply,
      address creator,
      uint8 communityFeeRatio,
      IBCTokenFactory.StakingConfig calldata stakingConfig
  ) external returns(uint256 targetETH, address pool) {
+     if (!IBCTokenFactory(factory).bcTokens(msg.sender)) revert Unauthorized();
+
      (address token0, address token1) =
          msg.sender < WETH ? (msg.sender, WETH) : (WETH, msg.sender);
```

# 3S-Launchpad-L02

## Two-tick-spacing gap in **_configureLiquidityRanges** leads to zero active liquidity at the seed price

| | |
|---|---|
| Id | 3S-Launchpad-L02 |
| Classification | Low |
| Impact | Low |
| Likelihood | High |
| Category | Bug |
| Status | Addressed in #75cd024. |

---

### Description

When a bonding curve token reaches its target market cap, **BCToken::seedLP** calls **LiquidityManager::seedLP** to create two concentrated liquidity positions on an Algebra pool: one containing only **token0** and another containing only **token1**. The tick ranges for these positions are computed by **LiquidityManager::_configureLiquidityRanges**, which determines boundaries relative to **baseSeedTick** (the seed tick rounded down to the nearest **TICK_SPACING** toward zero).

The current implementation offsets both positions away from **baseSeedTick** by one **TICK_SPACING** in each direction. Because **token0LowTick** is set to **baseSeedTick + TICK_SPACING** and **token1HighTick** is set to **baseSeedTick - TICK_SPACING**, there is a gap of two tick spacings (120 ticks) around the seed price where neither LP position provides liquidity. Since the seed tick always falls within this gap, the pool launches with no active liquidity at the current price. This means the first swaps after seeding experience higher slippage or may not execute until the price moves into an active range.

This behavior occurs regardless of whether the bonding curve token is **token0** or **token1**.

#### Steps to Reproduce

1. In **test/BondingCurve.t.sol**, the **setUp** function deploys a **BCToken**. The salt controls whether the deployed **BCToken** address is lower or higher than **WETH**, which determines the token ordering:

```
// To make BCToken = token0, use a fixed salt:
bytes32 salt = bytes32(uint256(123));
// To make BCToken = token1, use a generated salt:
bytes32 salt = _generateSalt();
```

```
coin = BCToken(
    factory.deploy(
-       "Init Token", "INIT", "Some generic description", "https://image.com", 50,
_generateSalt(),
+       "Init Token", "INIT", "Some generic description", "https://image.com", 50, salt,
        IBCTokenFactory.StakingConfig({deployStaking: true, alternativeFeeRecipient:
address(0)})
    )
);
```

2. Run the **test_successful_LPIsSeeded** test with verbosity to observe the tick values from event logs:

```
forge test --match-test test_successful_LPIsSeeded -vvvv
```

3. When BCToken is **token0** (**seedTick** = -202969, which falls between -202980 and -202920 when aligned to **TICK_SPACING** of 60):

- The **token1**-only LP position spans [-243780, -202980)
- The **token0**-only LP position spans [-202860, MAX_TICK)
- LP positions are inactive when the current tick is in [-202980, -202860), a gap of **two** tick spacings (120 ticks)

4. When BCToken is **token1** (**seedTick** = 202968, which falls between 202920 and 202980 when aligned to **TICK_SPACING** of 60):

- The **token1**-only LP position spans [MIN_TICK, 202860)
- The **token0**-only LP position spans [202980, 243780)
- LP positions are inactive when the current tick is in [202860, 202980), a gap of **two** tick spacings (120 ticks)

5. In both cases, the pool is initialized at **seedTick**, which falls inside the dead zone. The first swaps after seeding encounter zero active liquidity at the current tick.

---

### Recommendation

Adjust the tick boundaries so that one position extends to **baseSeedTick** itself rather than offsetting by an additional **TICK_SPACING**. The direction of the adjustment depends on whether **initialTick < seedTick** (BCToken is **token0**) or **initialTick > seedTick** (BCToken is **token1**):

```
 function _configureLiquidityRanges(int24 initialTick, int24 seedTick)
     internal
     pure
     returns (int24, int24, int24, int24)
 {
     if (initialTick == seedTick) revert InvalidLiquidityRanges();
     int24 baseSeedTick = _modulateTick(seedTick);
-    int24 token0LowTick = baseSeedTick + TICK_SPACING;
-    int24 token0HighTick =
-        _modulateTick(initialTick > seedTick ? initialTick : TickMath.MAX_TICK);
-    int24 token1LowTick =
-        _modulateTick(initialTick < seedTick ? initialTick : TickMath.MIN_TICK);
-    int24 token1HighTick = baseSeedTick - TICK_SPACING;
+    int24 token0LowTick;
+    int24 token0HighTick;
+    int24 token1LowTick;
+    int24 token1HighTick;
+
+    if (initialTick < seedTick) {
+        // BCToken is token0
+        token0LowTick = baseSeedTick;
+        token0HighTick = _modulateTick(TickMath.MAX_TICK);
+        token1LowTick = _modulateTick(initialTick);
+        token1HighTick = baseSeedTick - TICK_SPACING;
+    } else {
+        // BCToken is token1
+        token0LowTick = baseSeedTick + TICK_SPACING;
+        token0HighTick = _modulateTick(initialTick);
+        token1LowTick = _modulateTick(TickMath.MIN_TICK);
+        token1HighTick = baseSeedTick;
+    }
     if (token0LowTick >= token0HighTick || token1LowTick >= token1HighTick) {
         revert InvalidLiquidityRanges();
     }
     return (token0LowTick, token0HighTick, token1LowTick, token1HighTick);
 }
```

This reduces the dead zone from two tick spacings to one, which is the minimum gap required to keep the two positions non-overlapping while maintaining the single-token-per-position design.

# 3S-Launchpad-L03

Fee offset applied to virtual reserves instead of target ETH leads to incorrect pool initialization price and dead LP range

| | |
|---|---|
| Id | 3S-Launchpad-L03 |
| Classification | Low |
| Impact | Low |
| Likelihood | High |
| Category | Bug |
| Status | Addressed in #77c796f. |

---

## Description

The **LiquidityManager** contract computes two key prices when seeding an Algebra pool:

- **Seed price**: the bonding curve price at graduation, derived from **targetETH / initialSupply** (assuming BCToken is **token0**). This is used as the pool's initialization **sqrtPrice**.

- **Initial price**: the bonding curve price at deployment, derived from **virtualReserves / maxSupply** (assuming BCToken is **token0**). This defines the lower tick bound of the LP position because all circulating BCTokens remain in the pool above this price, meaning trading never occurs below it.

During graduation, **BCToken::_seedLP** deducts creator, protocol, and refund fees from the accumulated ETH balance before transferring the remaining WETH into the pool. These fees reduce the WETH that enters the pool, so both prices used to define the LP range must reflect the post-fee amounts.

However, **LiquidityManager::createAndInitializePool** applies the fee offset only to **virtualReserves** via **_offsetVirtualReserves** when computing the initial tick, and does not apply any offset to **targetETH** when computing the seed tick:

```
(uint160 seedSqrtX96Price, int24 seedTick) =
_computeSqrtPriceAndTick(msg.sender, token0, targetETH, initialSupply);
(, int24 initialTick) = _computeSqrtPriceAndTick(msg.sender, token0,
_offsetVirtualReserves(virtualReserves), maxSupply);
```

This produces two incorrect results:

1. **Incorrect pool initialization price**: The **seedSqrtX96Price** passed to **IAlgebraPool::initialize** uses the full **targetETH** without any fee deduction. Since **BCToken::_seedLP** deducts fees before sending WETH to the pool, the actual WETH entering the pool is less than **targetETH**, and the pool is initialized at a price higher than the true post-fee seed price.

2. **Asymmetric fee application breaks sell coverage**: Because the fee offset is applied only to one side of the range (initial price) but not the other (seed price), the LP range geometry becomes distorted. For a one-sided WETH position in range **[P_init, P_seed]**, the maximum sellable token amount is:

**max_sell = WETH / sqrt(P_init * P_seed)**

When the fee factor **(1-f)** is applied consistently to both prices:

- WETH in pool: **W = (T - V) * (1 - f)** (fees deducted from real ETH only)

- Seed price: **P_seed = T * (1-f) / S0**

- Initial price: **P_init = V * (1-f) / Smax**

Substituting and simplifying: **max_sell = (T-V) * sqrt(Smax * S0) / sqrt(V * T)**, where the **(1-f)** factor cancels out entirely, meaning the sellable supply is fee-independent.

However, the current code applies the offset asymmetrically (only to **virtualReserves**), so the cancellation does not occur. This results in only ~94% of the circulating supply being sellable back into the LP (assuming a 1B total supply and 150M tokens seeded into the LP, the minimum tick is reached at 800M tokens sold).

Additionally, **_offsetVirtualReserves** hardcodes the total fee as 25% (**2_500 / 10_000**), but the actual fees are derived from per-token **CREATOR_FEE**, **PROTOCOL_FEE**, and a refund fee equal to half the creator fee, which may not sum to 25%.

```
function _offsetVirtualReserves(uint256 _virtualReserves) internal pure returns (uint256) {
    return _virtualReserves - _virtualReserves * 2_500 / 10_000;
}
```

#### Steps to Reproduce

1. A **BCToken** is deployed with **CREATOR_FEE** and **PROTOCOL_FEE** values that do not sum (with the refund fee) to the hardcoded 25%.

2. **LiquidityManager::createAndInitializePool** computes **seedSqrtX96Price** using the raw **targetETH** (no fee offset) and passes it to **IAlgebraPool::initialize**, setting the pool's price higher than the actual post-fee ratio.

3. The initial tick is computed with **_offsetVirtualReserves(virtualReserves)** but the seed tick is not offset, creating an asymmetric range that caps sellable supply at ~94% of circulating tokens.

4. At graduation, **BCToken::_seedLP** deducts fees from **ethBalance** and sends the remaining WETH to the pool. The pool price does not match the actual token/WETH ratio, and the LP range does not allow full sell coverage of the circulating supply.

---

### Recommendation

Apply the fee offset to **both targetETH** and **virtualReserves** using the actual fee parameters. This ensures the **(1-f)** factor cancels out in the max-sell formula, preserving full sell coverage regardless of fee configuration.

**1. Add `protocolFee` and `creatorFee` parameters to `LiquidityManager::createAndInitializePool`:**

```
  function createAndInitializePool(
      uint256 virtualReserves,
      uint256 initialSupply,
      uint256 maxSupply,
      address creator,
-     uint8 communityFeeRatio
+     uint8 communityFeeRatio,
+     uint80 protocolFee,
+     uint80 creatorFee,
  ) external returns(uint256 targetETH, address pool);
```

**2. Compute `totalFeeBps` and offset both values in `createAndInitializePool`:**

```
  targetETH = ((maxSupply - initialSupply) * virtualReserves) / initialSupply +
virtualReserves;
+ uint256 totalFeeBps = uint256(protocolFee) + uint256(creatorFee) +
uint256(creatorFee) / 2;
- (uint160 seedSqrtX96Price, int24 seedTick) =
_computeSqrtPriceAndTick(msg.sender, token0, targetETH, initialSupply);
- (, int24 initialTick) = _computeSqrtPriceAndTick(msg.sender, token0,
_offsetVirtualReserves(virtualReserves), maxSupply);
+ (uint160 seedSqrtX96Price, int24 seedTick) =
_computeSqrtPriceAndTick(msg.sender, token0, _offsetByFees(targetETH,
totalFeeBps), initialSupply);
+ (, int24 initialTick) = _computeSqrtPriceAndTick(msg.sender, token0,
```

_offsetByFees(virtualReserves, totalFeeBps), maxSupply);

3. Apply the same correction in `LiquidityManager::seedLP`:

```
- (, int24 initialTick) = _computeSqrtPriceAndTick(token, token0,
_offsetVirtualReserves(coin.VIRTUAL_BALANCE()), coin.MAX_SUPPLY());
+ uint256 totalFeeBps = uint256(coin.PROTOCOL_FEE()) +
uint256(coin.CREATOR_FEE()) + uint256(coin.CREATOR_FEE()) / 2;
+ (, int24 initialTick) = _computeSqrtPriceAndTick(token, token0,
_offsetByFees(coin.VIRTUAL_BALANCE(), totalFeeBps), coin.MAX_SUPPLY());
```

4. Replace `_offsetVirtualReserves` with a generic `_offsetByFees`:

```
- function _offsetVirtualReserves(uint256 _virtualReserves) internal pure returns
(uint256) {
-     return _virtualReserves - _virtualReserves * 2_500 / 10_000;
- }
+ function _offsetByFees(uint256 _value, uint256 _totalFeeBps) internal pure returns
(uint256) {
+     return _value - _value * _totalFeeBps / 10_000;
+ }
```

5. Update `BCToken` to pass fees to `createAndInitializePool`:

```
  (TARGET_ETH, lp) =
ILiquidityManager(LIQUIDITY_MANAGER).createAndInitializePool(
      params._virtualReserves,
      params._initialSupply,
      params._maxSupply,
      params._creator,
-     params._communityFeeRatio
+     params._communityFeeRatio,
+     params._protocolFee,
+     params._creatorFee,
  );
```

# 3S-Launchpad-L04

## Unoverridden **renounceOwnership** in **BCTokenFactory** leads to permanent denial of service on all pool swaps

| | |
|---|---|
| Id | 3S-Launchpad-L04 |
| Classification | Low |
| Impact | Medium |
| Likelihood | Low |
| Category | Bug |
| Status | Addressed in #94cbb3c. |

### Description

**BCTokenFactory** inherits OpenZeppelin's **Ownable** without overriding the **renounceOwnership** function. The factory's **owner()** is queried dynamically during every swap through the plugin fee collection flow. Specifically, **AlgebraFeePluginFactoryV1::factoryOwner()** returns **ISimplifiedTokenFactory(BC_TOKEN_FACTORY).owner()**, and **AlgebraFeePluginV1::handlePluginFee** reads this value to determine the **protocolOwner** address that should receive protocol fees.

When **staking** is not configured for a given pool (i.e., **staking == address(0)**), **handlePluginFee** executes **IERC20(coin).transfer(protocolOwner, coinAmount)** directly. If **renounceOwnership** has been called, **protocolOwner** resolves to **address(0)**, and standard ERC20 implementations revert on transfers to the zero address. Since **handlePluginFee** is invoked as a before-swap hook, this reversion propagates up and blocks all swaps on affected pools.

The plugin is immutably bound to the pool at deployment time and cannot be replaced. The only recovery path is to disable the before-swap hook via **setPluginConfig**, which removes the plugin fee mechanism entirely. Additionally, all **onlyFactoryOwner**-gated functions on **AlgebraFeePluginFactoryV1** (such as **setProtocolFeeRatio**) become permanently inaccessible once ownership is renounced, since no **msg.sender** can satisfy the **owner()** check against **address(0)**.

### Recommendation

Override **renounceOwnership** in **BCTokenFactory** to explicitly revert, preventing accidental or intentional ownership renunciation.

```
+ function renounceOwnership() public pure override {
+     revert();
+ }
```

# 3S-Launchpad-N01

Redundant arithmetic in **targetETH** calculation leads to unnecessary gas consumption

| | |
|---|---|
| Id | 3S-Launchpad-N01 |
| Classification | None |
| Category | Suggestion |
| Status | Addressed in #e04e79d. |

---

## Description

**LiquidityManager::createAndInitializePool** computes **targetETH**, the amount of ETH required to fill the bonding curve, using the following formula:

**targetETH = ((maxSupply - initialSupply) * virtualReserves) / initialSupply + virtualReserves;**

This expression performs a subtraction, two multiplications, a division, and an addition. It can be algebraically simplified to:

**targetETH = (maxSupply * virtualReserves) / initialSupply;**

Both formulas produce identical results in Solidity integer arithmetic because the **+ virtualReserves** term is equivalent to adding **initialSupply * virtualReserves / initialSupply**, which is exactly divisible and does not recover any truncation from the prior division. The simplified form reduces the operation count from five to three (one multiplication and one division), saving gas on every call.

---

## Recommendation

Simplify the **targetETH** calculation:

```
- targetETH = ((maxSupply - initialSupply) * virtualReserves) / initialSupply + virtualReserves;
+ targetETH = (maxSupply * virtualReserves) / initialSupply;
```

# 3S-Launchpad-N02

Use of single-step **Ownable** in **BCTokenFactory** allows irrecoverable loss of ownership through accidental transfer

| | |
|---|---|
| Id | 3S-Launchpad-N02 |
| Classification | None |
| Category | Suggestion |
| Status | Acknowledged |

## Description

**BCTokenFactory** is the central administrative contract of the protocol. It inherits from OpenZeppelin's **Ownable** and gates every privileged operation behind the **onlyOwner** modifier. These operations include setting the bonding curve (**setBondingCurve**), the liquidity manager (**setLiquidityManager**), the harvester (**setHarvester**), updating fee parameters (**updateFees**), tuning initial token supply curves (**updateInitialParams**), pausing the entire protocol (**setProtocolPaused**), withdrawing collected ETH (**withdraw**), and configuring the token vesting address (**setTokenVesting**). The owner address is also forwarded as the LayerZero **_delegate** for every newly deployed **BCToken** via **BCTokenFactory::_deployToken**. Because **Ownable** exposes a single-step **transferOwnership** function, a transfer to an incorrect or inaccessible address would permanently lock out all administrative control over the factory and, by extension, over the protocol's fee structure, pause mechanism, and future deployments.

## Recommendation

Replace **Ownable** with OpenZeppelin's **Ownable2Step**, which requires the proposed new owner to explicitly call **acceptOwnership** before the transfer takes effect, ensuring the receiving address is valid and controlled.

# 3S-Launchpad-N03

## Redundant arithmetic in **_distributeFee** leads to unnecessary gas overhead

| | |
|---|---|
| Id | 3S-Launchpad-N03 |
| Classification | None |
| Category | Suggestion |
| Status | Addressed in #e71bc1c. |

---

### Description

**AlgebraFeePluginV1::_distributeFee** splits a given fee **amount** between two recipients according to a **ratio** parameter expressed as a percentage in the range [0, 100]. The share for **recipientA** is computed on line 94 as **uint256 amountA = (amount * ratio * 100) / 10_000;**. Multiplying by **100** and then dividing by **10_000** is mathematically identical to dividing by **100** alone. The extra multiplication introduces a superfluous operation that increases gas cost on every fee distribution and reduces readability without any change in the result.

---

### Recommendation

Simplify the expression by removing the unnecessary *** 100** factor and dividing directly by **100**:

```
-       uint256 amountA = (amount * ratio * 100) / 10_000;
+       uint256 amountA = (amount * ratio) / 100;
```